String Matching

Bin Wang

School of Software
Tsinghua University

December 16, 2009

Outline

1. **The String Matching problem**
   - Overview
   - Notation and terminology
2. **The String Matching Algorithms**
   - The naïve string-matching algorithm
   - The Rabin-Karp algorithm
   - String Matching with finite automata
   - The Knuth-Morris-Pratt algorithm
   - The Boyer-Moore algorithm

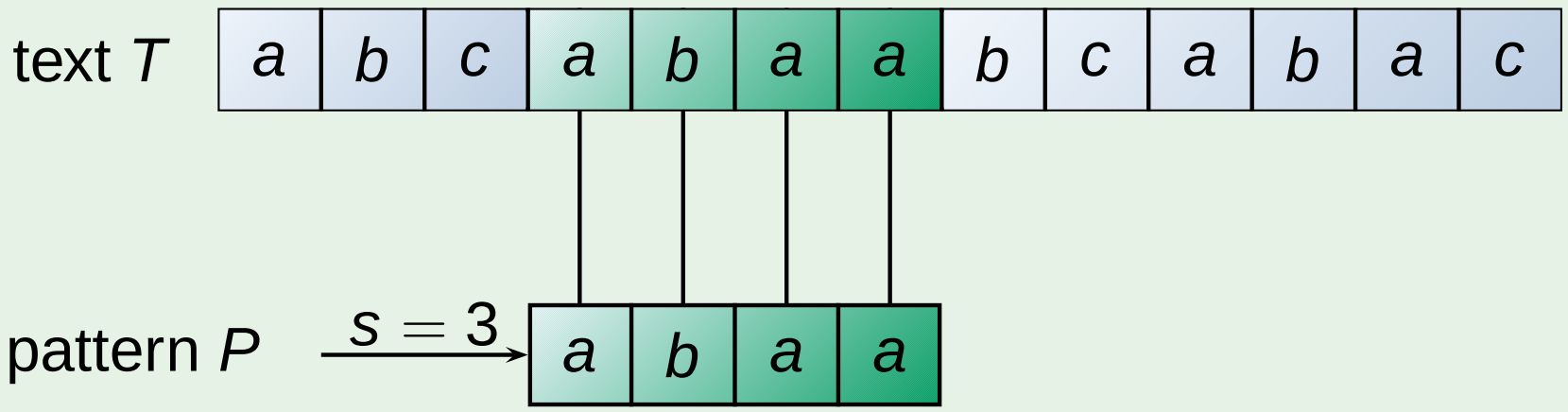
The string-matching problem

Definition

- The ***string-matching problem*** is to find all occurrences of the pattern $P[1..m]$ in the text $T[1..n]$.
- We further assume that the elements of P and T are characters drawn from a finite alphabet $\sum$. For example, we may have $\sum = \{0, 1\}$ or $\sum = \{a, b, \ldots, z\}$.
- We say that pattern P occurs with shift s in text T if $0 \leq s \leq n - m$ and $T[s + 1..s + m] = P[1..m]$.

The string-matching problem

Example



The string-matching algorithms

Algorithms comparison

Algorithm	Preprocessing	Matching time
Naive	0	$O((n-m+1)m)$
Rabin-Karp	$\Theta(m)$	$O((n-m+1)m)$
Finite automaton	$O(m\|\sum\|)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$
Boyer-Moore	$\Theta(m+\|\sum\|)$	$\Omega(n/m), O(n)$
Shift-Or	$\Theta(m+\|\sum\|)$	$\Theta(n)$

Notation and terminology

- $\sum$: a finite alphabet.
- $\sum^*$: the set of all finite-length strings formed using characters from $\sum$.
- concatenation of of two strings x and y : xy, has length $|x| + |y|$ and consists of the characters from x followed by the characters from y.
- prefix of a string, denoted $w \sqsubset x$, if $x = wy$ for some string $y \in \sum^*$.
- suffix of a string, denoted $w \sqsupset x$, if $x = yw$ for some string $y \in \sum^*$.

Notation and terminology

Example

$ab \sqsubset abcca$, $cca \sqsupset abcca$, $x \sqsupset y \iff xa \sqsupset ya$

Using the notation

- We denote the k-character prefix $P[1..k]$ of the pattern $P[1..m]$ by P_k. Thus, $P_0 = \varepsilon$ and $P_m = P = P[1..m]$. Similarly, we denote the k-character prefix of the text T as T_k .
- Using this notation, we can state the string-matching problem as that of finding all shifts s in the range $0 \leq s \leq n - m$ such that $P \sqsupset T_{s+m}$.

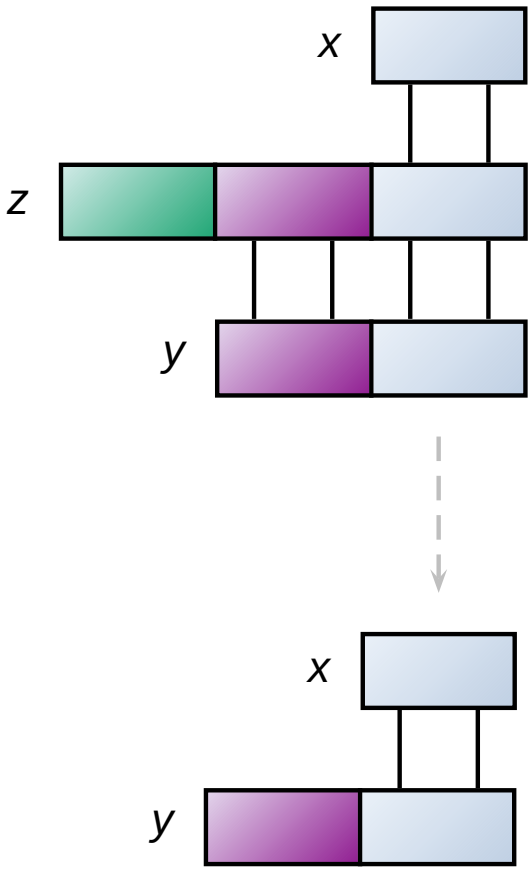
Overlapping-suffix lemma

Lemma 32.1 (Overlapping-suffix lemma)

Suppose that x, y, and z are strings such that $x \sqsupset z$ and $y \sqsupset z$. If $|x| \leq |y|$, then $x \sqsupset y$. If $|x| \geq |y|$, then $y \sqsupset x$. If $|x| = |y|$, then $x = y$.

Skip Proof

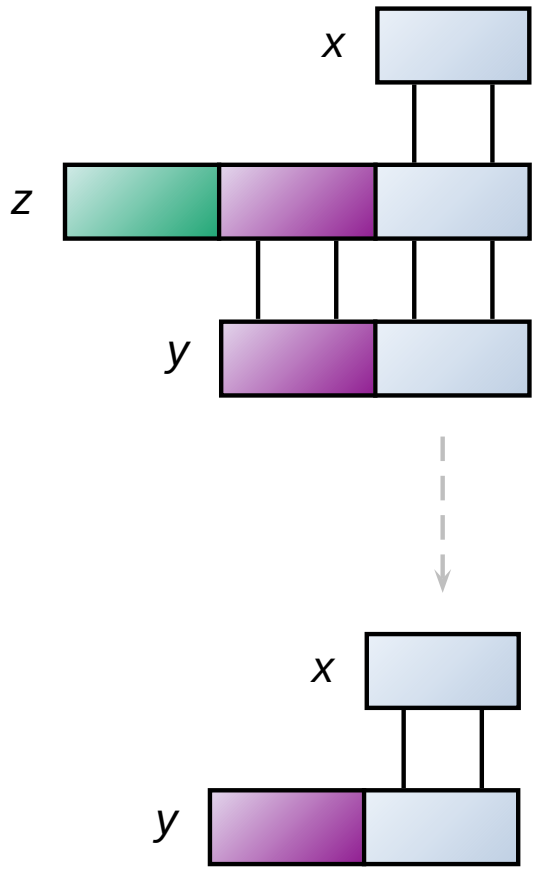
Overlapping-suffix lemma



(a) if $|x| \leq |y|$,

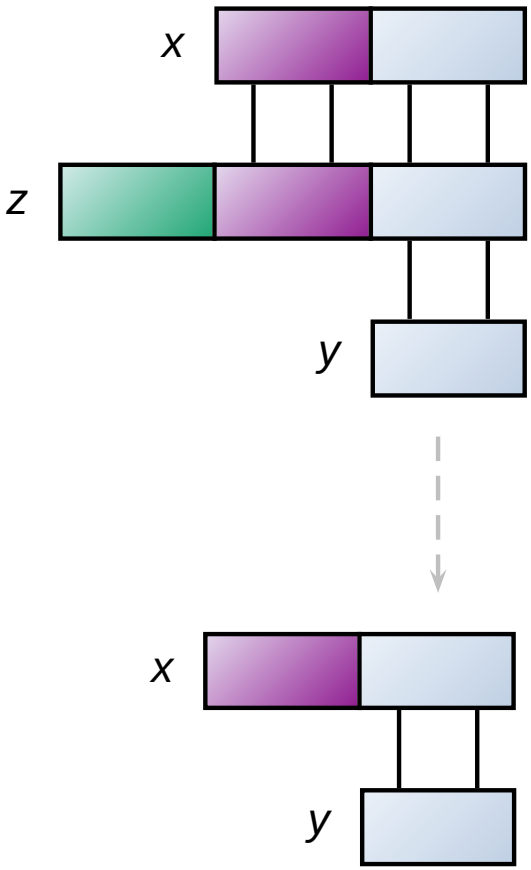
then $x \sqsupset y$.

Overlapping-suffix lemma



(a) if $|x| \leq |y|$,

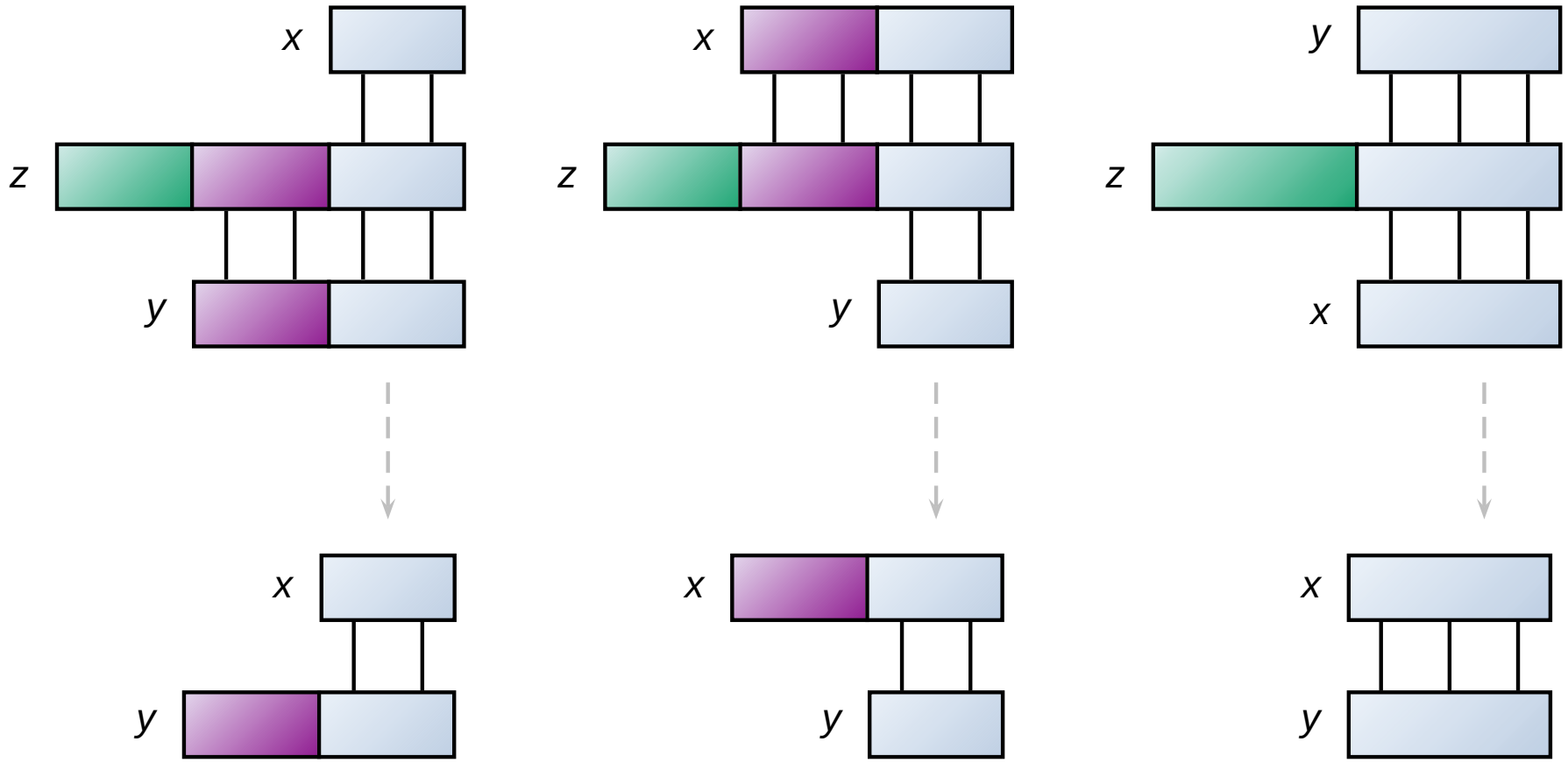
then $x \sqsupset y$.



(b) if $|x| \geq |y|$,

then $y \sqsupset x$.

Overlapping-suffix lemma



(a) if $|x| \leq |y|$, then $x \sqsupset y$.

(b) if $|x| \geq |y|$, then $y \sqsupset x$.

(c) if $|x| = |y|$, then $x = y$.

The naïve string-matching algorithm

NAIVE-STRING-MATCHER(T, P)

```
n ← length[T]
m ← length[P]
for s ← 0 to n − m
      do if P[1..m] = T[s + 1..s + m]
            then print "Pattern occurs with shift" s
```

Running time

$$\Theta((n - m + 1)m)$$

The Rabin-Karp algorithm–Basic idea

Intuition

- Let us assume that $\sum = \{0, 1, 2, \ldots, 9\}$.
- Given a pattern $P[1..m]$, let p denote its corresponding decimal value.
  Example: $31415 \Rightarrow p = 31,415$
- Given a pattern $P[1..m]$, we let p denote its corresponding decimal value. In a similar manner, given a text $T[1..n]$, let t_s denote the decimal value of the length-m substring $T[s+1..s+m]$. Thus
  $$T[s+1..s+m] = P[1..m] \iff t_s = p$$

The Rabin-Karp algorithm–Basic idea

Running time

- We can compute p in time $\Theta(m)$ using Horner's rule.
  $p = P[m] + 10(P[m-1] + 10(P[m-2] + \cdots + 10(P[2] + 10P[1]) \cdots))$
- Compute all the t_s values in $\Theta(n - m + 1)$.
  $t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1]$.

Example

$m = 5, t_s = 31415, T[s+5+1] = 2$
$t_{s+1} = 10(31415 - 10000 \cdot 3) + 2 = 14152$

The Rabin-Karp algorithm–Improvement

Use modulus

Compute p and t_s's modulo by choosing a suitable modulus q.

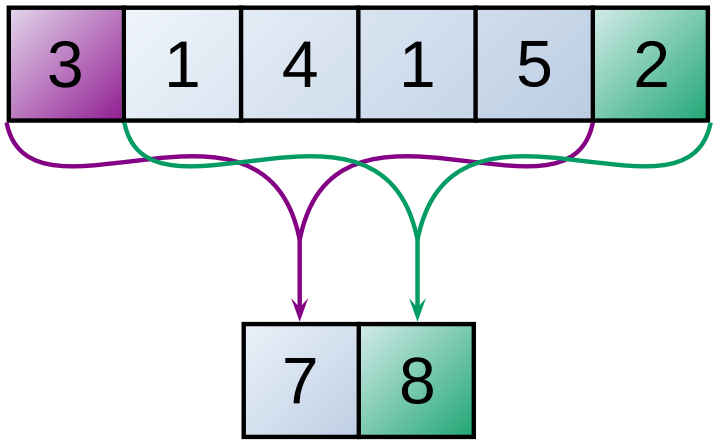
$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$$

The Rabin-Karp algorithm–Improvement

Use modulus

Compute p and t_s's modulo by choosing a suitable modulus q.

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$$

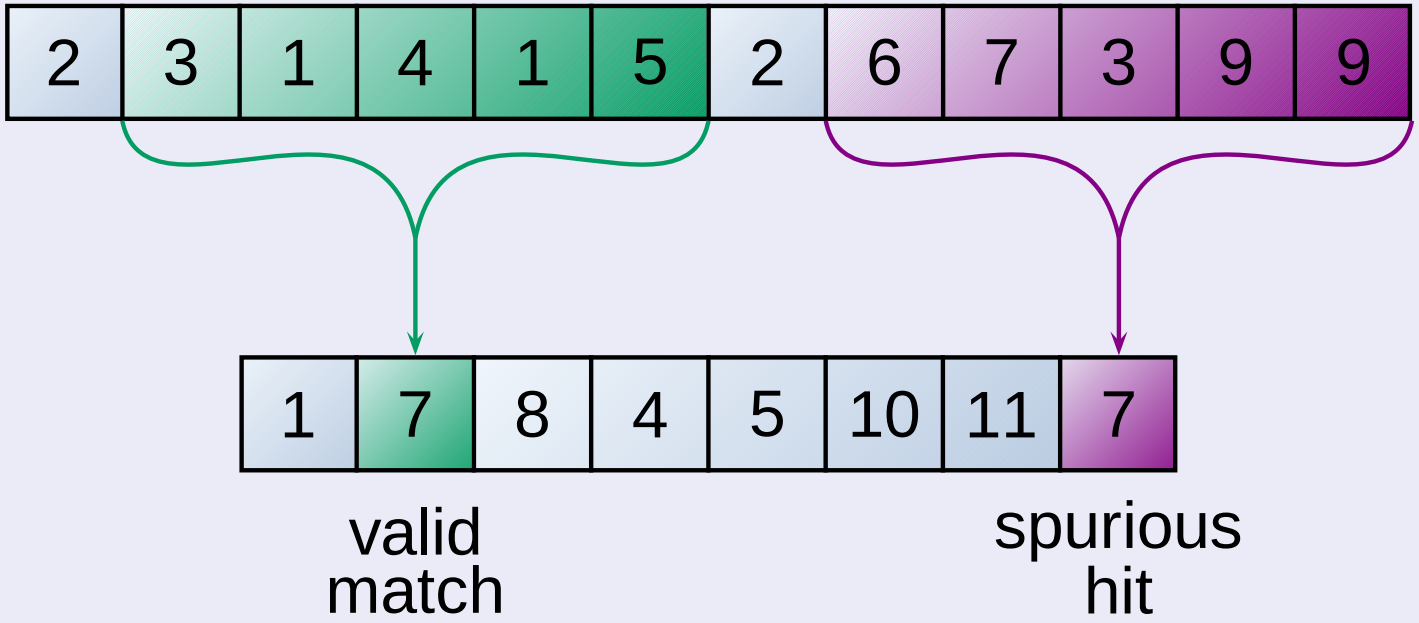


Example

$$14152 \equiv 10 \cdot (7 - 3 \cdot 3) + 2 \pmod{13}$$
$$\equiv 8 \pmod{13}$$

The Rabin-Karp algorithm–Improvement

Spurious hit



RABIN-KARP-MATCHER

RABIN-KARP-MATCHER

RABIN-KARP-MATCHER(T, P, d, q)

```
n ← length[T]
m ← length[P]
h ← d^(m−1) mod q
p ← 0
t_0 ← 0
for i ← 1 to m                    ▷ Preprocessing
      do p ← (dp + P[i]) mod q
         t_0 ← (dt_0 + T[i]) mod q
```

RABIN-KARP-MATCHER

RABIN-KARP-MATCHER (Cont.)

```
gorithm 9   for s ← 0 to n − m          ▷ Matching
  10            do if p = t_s
  11                  then if P[1..m] = T[s + 1..s + m]
  12                          then print "Pattern occurs
                                      with shift" s
  13                  if s < n − m
  14                  then t_{s+1} ← (d(t_s − T[s + 1]h)
                                  + T[s + m + 1]) mod q
```

RABIN-KARP-MATCHER

Running time

- If $P = a^m$ and $T = a^n$, then the verifications take time $\Theta((n - m + 1)m)$, since each of the $n - m + 1$ possible shifts is valid.
- In many applications, we expect few valid shifts and $O(n/q)$ spurious hits.
- The expected matching time: $O(n) + O(m(v + n/q))$, where v is the number of valid shifts.
- If $v = (O(1))$ and we choose $q \geq m$, the expected matching time is $O(n)$.

Finite automata

Definition

A ***finite automaton*** M is a 5-tuple $(Q, q_0, A, \sum, \delta)$, where

- Q is a finite set of ***states***,
- $q_0 \in Q$ is the ***start states***,
- $A \subseteq Q$ is a distinguished set of ***accepting states***,
- $\sum$ is a finite ***input alphabet***,
- δ is a function from $Q \times \sum$ into Q, called the ***transition function*** of M.

Finite automata

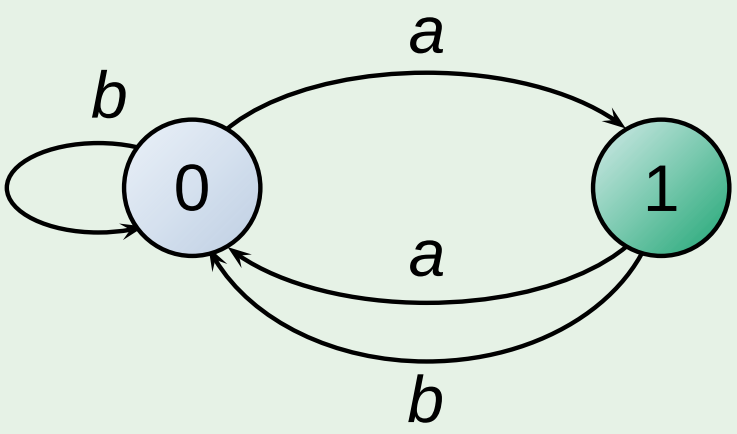
Definition

- If the auomaton is in state q and reads input character a, it moves from state q to state $\delta(q, a)$.
- If its current state q is a member of A, the machine M is said to have accepted the string read so far.
- A finite automaton M induces a function ϕ, called **final-sate function**.
  $\phi(\varepsilon) = q_0$
  $\phi(wa) = \delta(\phi(w), a)$, for $w \in \sum^*, a \in \sum$.

Finite automata

Example

state	input a	input b
0	1	0
1	0	0



$A = \{1\}$.

ϕ(abaaa) = 1 : accepted, ϕ(abbaa) = 0 : rejected.

String-matching automata

String-matching automaton to a given pattern $P[1..m]$

- $M = (Q, q_0, A, \sum, \delta)$,
- $Q = \{0, 1, \ldots, m\}$,
- $q_0 = 0, A = \{m\}$,

String-matching automata

String-matching automaton to a given pattern $P[1..m]$

- **suffix function** of $P : \sigma(x) = \max\{k : P_k \sqsupset x\}$.
  $P = ab, \sigma(\varepsilon) = 0, \sigma(ccaca) = 1$, and $\sigma(ccab) = 2$.
  For a pattern P of length m, we have $\sigma(x) = m \iff P \sqsupset x$,
- $\delta(q, a) = \sigma(P_q a)$.

String-matching automata

Example

$P = ababaca$:

$\delta(0, a) = \sigma(P_0 a) = \sigma(a) = 1,$
$\delta(0, b) = \sigma(P_0 b) = \sigma(b) = 0,$
$\delta(0, c) = \sigma(P_0 c) = \sigma(c) = 0,$

$\delta(1, a) = \sigma(P_1 a) = \sigma(aa) = 1,$
$\delta(1, b) = \sigma(P_1 b) = \sigma(ab) = 2,$
$\delta(1, c) = \sigma(P_1 c) = \sigma(ac) = 0,$

String-matching automata

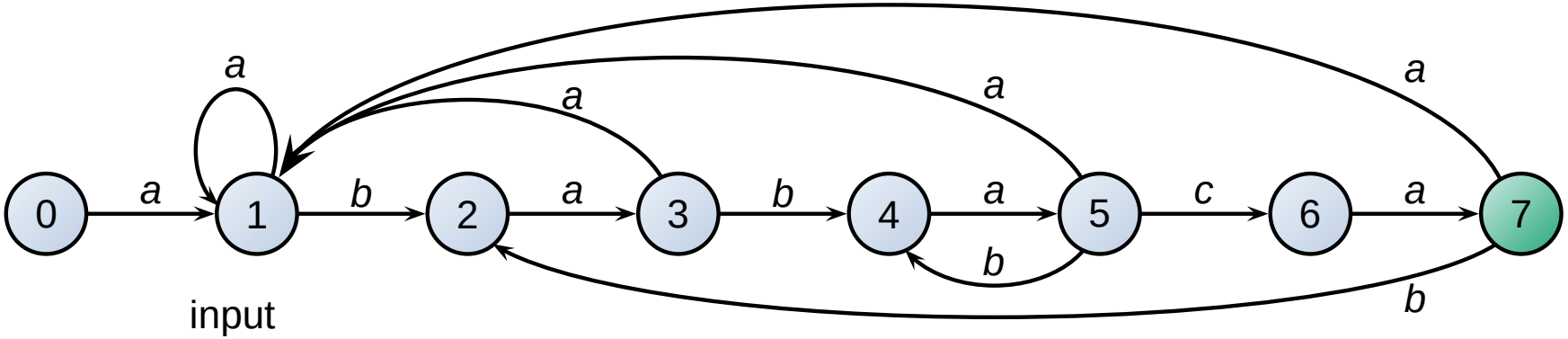
Example

$P = ababaca$:

$\delta(2, a) = \sigma(P_2 a) = \sigma(aba) = 3,$
$\delta(2, b) = \sigma(P_2 b) = \sigma(abb) = 0,$
$\delta(2, c) = \sigma(P_2 c) = \sigma(abc) = 0,$

$\delta(3, a) = \sigma(P_3 a) = \sigma(abaa) = 1,$
$\delta(3, b) = \sigma(P_3 b) = \sigma(abab) = 4,$
$\delta(3, c) = \sigma(P_3 c) = \sigma(abac) = 0.$

String-matching automata



state	input a	b	c	P
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

i	–	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	–	a	b	a	b	a	b	a	c	a	b	a
state $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

String-matching automata

Finite automaton matcher

FINITE-AUTOMATON-MATCHER(T, δ, m)

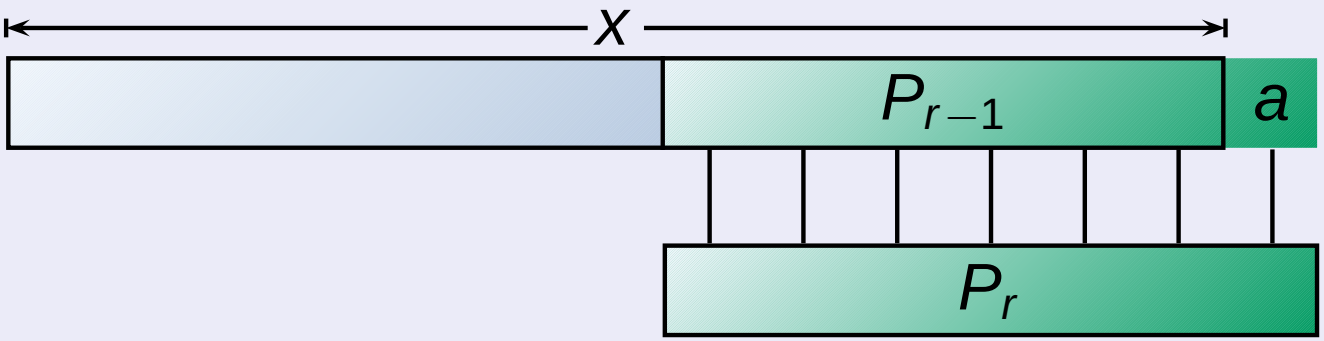
```
n ← length[T]
q ← 0
for i ← 1 to n
        do q ← δ(q, T[i])
           if q = m
              then print "Pattern ocurs
              with shift" i − m
```

Correctness of FINITE-AUTOMATON-MATCHER

Lemma 32.2 (suffix-function inequality)

For any string x and character a, we have $\sigma(xa) \leq \sigma(x) + 1$.

Proof.



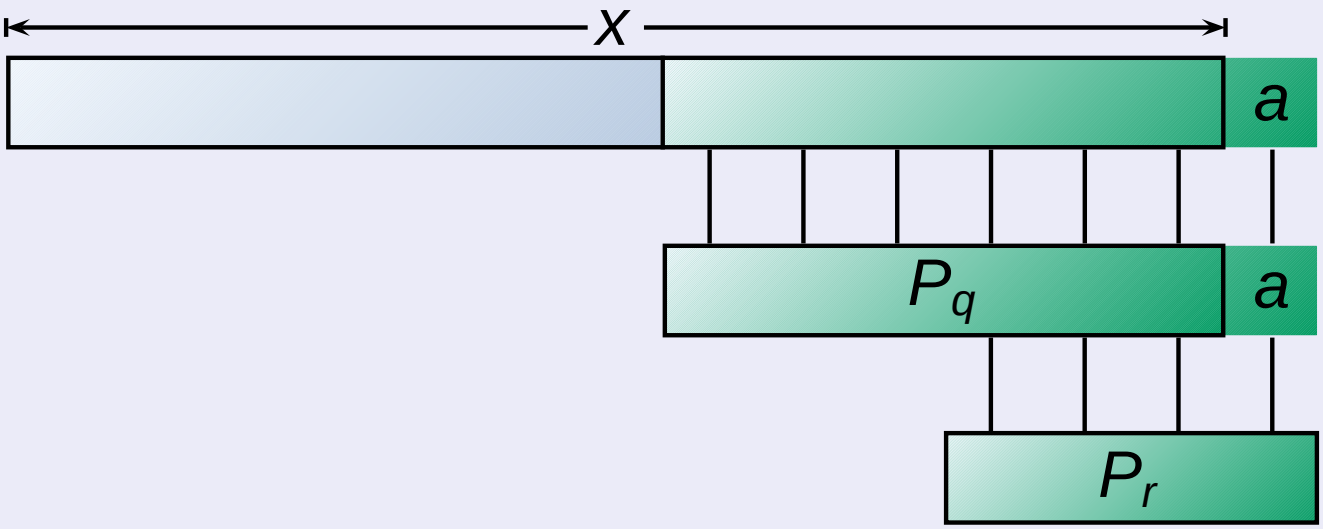
□

Correctness of FINITE-AUTOMATON-MATCHER

Lemma 32.3 (suffix-function recursion lemma)

For any string x and character a, if $q = \sigma(x)$, then $\sigma(xa) = \sigma(P_q a)$

Proof.



□

Correctness of FINITE-AUTOMATON-MATCHER

Theorem 32.4

If ϕ is the final-state function of a string-matching automaton for a given pattern P and $T[1..n]$ is an input text for the automaton, the $\phi(T_i) = \sigma(T_i)$,for $i = 0, 1, \ldots, n$.

Correctness of FINITE-AUTOMATON-MATCHER

Proof.

$$
\begin{aligned}
\phi(T_{i+1}) &= \phi(T_i a) && \text{(by the definition of } T_{i+1}\text{)} \\
&= \delta(\phi(T_i), a) && \text{(by the definition of } \phi\text{)} \\
&= \delta(q, a) && \text{(by the definition of } a\text{)} \\
&= \sigma(P_q a) && \text{(by the definition of } \delta\text{)} \\
&= \sigma(T_i a) && \text{(by Lemma 32.3)} \\
&= \sigma(T_{i+1})
\end{aligned}
$$

□

Computing the transition function

COMPUTE-TRANSITION-FUNCTION$(P, \sum)$

```
m ← length[P]
for q ← 0 to m
      do for each character a ∈ Σ
            do k ← min(m + 1, q + 2)
               repeat k ← k − 1
                  until P_k ⊐ P_q a
               δ(q, a) ← k
return δ
```

The prefix function

Basic Idea

The **prefix function** for a pattern encapsulates knowledge about how the pattern matches against shifts of itself.

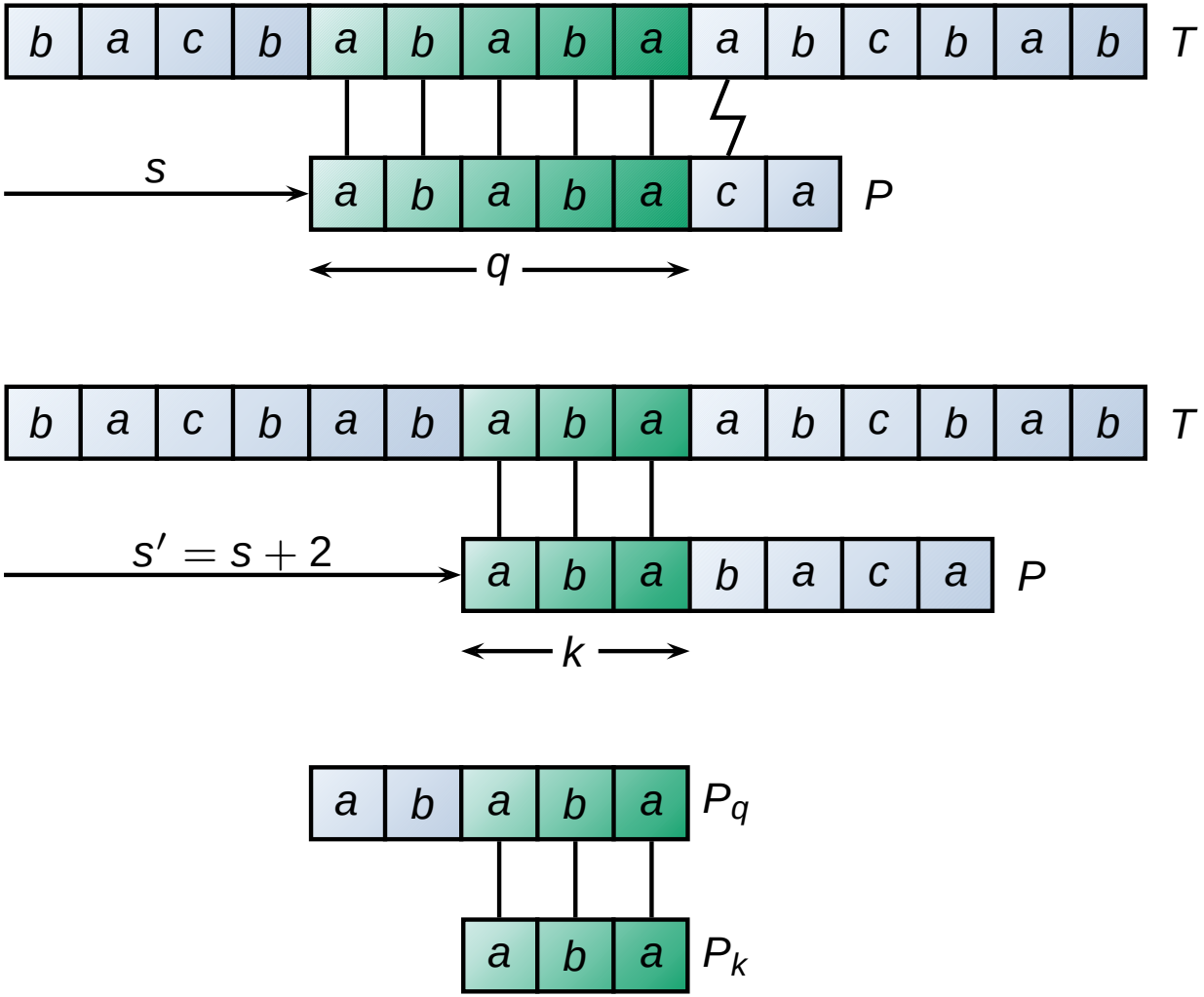
- Avoid testing useless shifts in the naïve pattern-matching algorithm.
- Avoid the pre-computation of δ for a string-matching automaton.

The prefix function

Basic Idea

- Given that pattern $P[1..q]$ match text characters $T[s+1..s+q]$, what is the least shift $s' > s$ such that $P[1..k] = T[s'+1..s'+k]$, where $s'+k = s+q$?

The prefix function



The prefix function

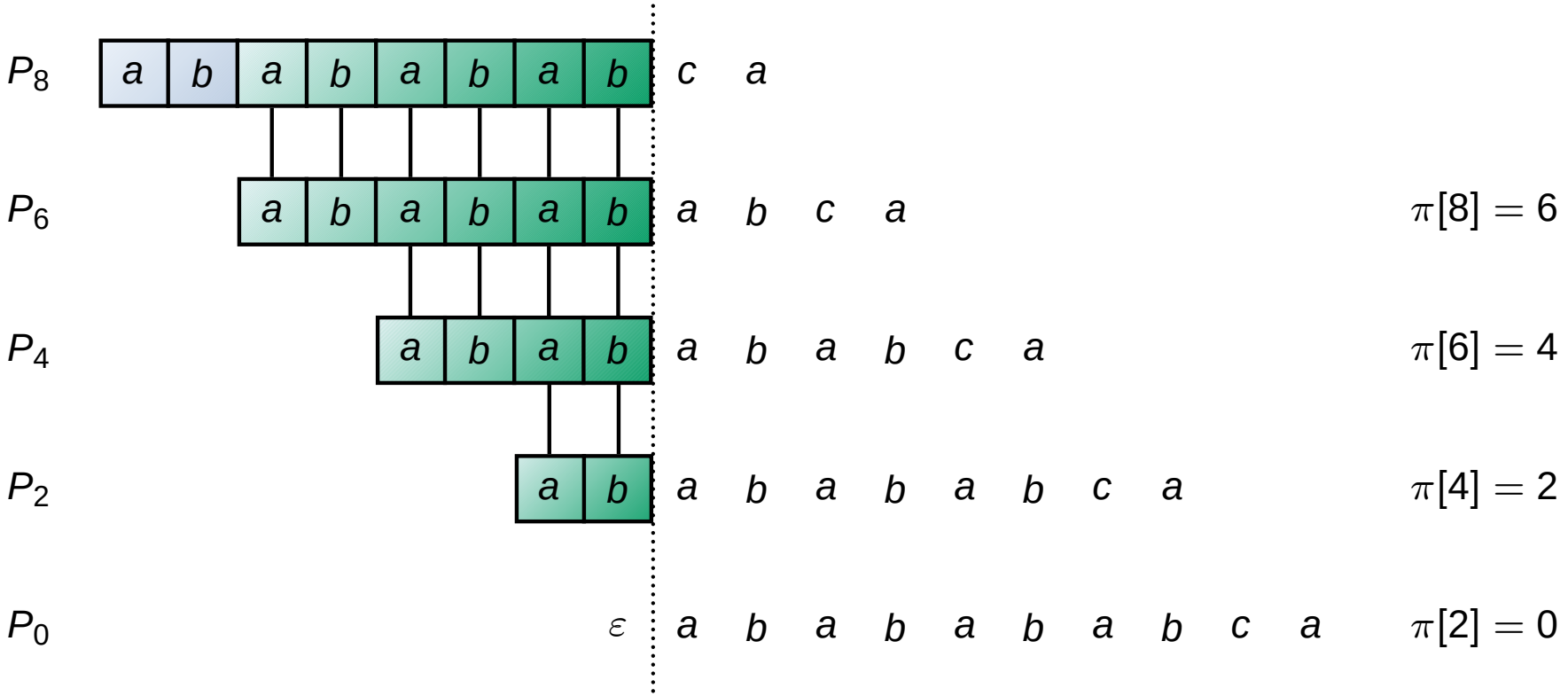
Basic Idea

- Given a pattern $P[1..m]$, the **prefix function** for the pattern P is the function $\pi : \{1, 2, \ldots, m\} \to \{0, 1, \ldots, m-1\}$ such that

$$\pi[q] = \max\{k : k < q \text{ and } P_k \sqsupset P_q\}.$$

The prefix function

i	1	2	3	4	5	6	7	8	9	10
$P[i]$	a	b	a	b	a	b	a	b	c	a
$\pi[i]$	0	0	1	2	3	4	5	6	0	1



KMP-MATCHER

KMP-MATCHER(T, P)

$n \leftarrow length[T]$
$m \leftarrow length[P]$
$\pi \leftarrow$ COMPUTE-PREFIX-FUNCTION(P)
$q \leftarrow 0$ $\triangleright$ Number of characters matched.

KMP-Matcher

```
ithm 5   for i ← 1 to n                    ▷ Scan the text from left to right.
     6     do while q > 0 and P[q + 1] ≠ T[i]
     7         do q ← π[q]                 ▷ Next character does not match.
     8       if P[q + 1] = T[i]
     9         then q ← q + 1              ▷ Next character matches.
    10       if q = m                      ▷ Is all of P matched?
    11         then print "Pattern ocurs with shift" i − m
    12           q ← π[q]                  ▷ Look for the next match.
```

KMP-Matcher

Compute-Prefix-Function(P)

```
m ← length[P]
π[1] ← 0
k ← 0
for q ← 2 to m
      do while k > 0 and P[k + 1] ≠ P[q]
            do k ← π[k]
         if P[k + 1] = P[q]
            then k ← k + 1
         π[q] ← k
return π
```

KMP-MATCHER

Running-time

Analysis of COMPUTE-PREFIX-FUNCTION:

- We associate a potential of k with the current state k of the algorithm.
- The running time is $\Theta(m)$.

Analysis of KMP-MATCHER:

- We use the value of q as the potential function.
- The running time is $\Theta(n)$.

Correctness of the prefix-function

Definition

Let $\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \ldots, \pi^{(t)}[q]\}$, where $\pi^*[q]$ is defined in terms of functional iteration, so that $\pi^{(0)}[q] = q$ and $\pi^{(i+1)}[q] = \pi[\pi^{(i)}[q]]$ for $i \geq 1$. Example

Lemma 32.5 (Prefix-function iteration lemma)

Let P be a pattern of length m with prefix function π. Then for $q = 1, 2, \ldots m$, we have $\pi^*[q] = \{k : k < q \text{ and } P_k \sqsupset P_q\}$.

Correctness of the prefix-function

Proof.

- We first prove that $i \in \pi^*[q]$ implies $P_i \sqsupset P_q$. Therefore $\pi^*[q] \subseteq \{k : k < q \text{ and } P_k \sqsupset P_q\}$.
- We prove that $\{k : k < q \text{ and } P_k \sqsupset P_q\} \subseteq \pi^*[q]$ by contradiction. Suppose to the contrary that there is an integer in the set $\{k : k < q \text{ and } P_k \sqsupset P_q\} - \pi^*[q]$, and let j be the largest such value. We must have $j < \pi[q]$.

Correctness of the prefix-function

Proof.

- We let j' denote the smallest integer in $\pi^*[q]$ that is greater than j. We have $P_j \sqsupset P_q$, and we have $P_{j'} \sqsupset P_q$. Thus $P_j \sqsupset P_{j'}$ by Lemma32.1, and j is the largest value less than j' with this property.

  Therefore, we must have $\pi[j'] = j$, since $j' \in \pi^*[q]$, we mush have $j \in \pi^*[q]$ as well.

$\square$

Correctness of the prefix-function

Lemma 32.6

Let P be a pattern of length m, and let π be the prefix function for P. For $q = 1, 2, \ldots m$, if $\pi[q] > 0$, then $\pi[q] - 1 \in \pi^*[q-1]$.

Proof.

If $r = \pi[q] > 0$, then $r < q$ and $P_r \sqsupset P_q$; thus, $r - 1 < q - 1$ and $P_{r-1} \sqsupset P_{q-1}$(by dropping the last character from P_r and P_q). By Lemma 32.5, therefore, $\pi[q] - 1 = r - 1 \in \pi^*[q-1]$. $\square$

Correctness of the prefix-function

Corollary 32.7

Let P be a pattern of length m, and let π be the prefix function for P. For $q = 2, 3, \ldots m$,

$$\pi[q] = \begin{cases} 0 & \text{if } E_{q-1} = \emptyset \\ 1 + \max\{k \in E_{q-1}\} & \text{if } E_{q-1} \neq \emptyset \end{cases}$$

where E_{q-1} is the subset of $\pi^*[q-1]$ for $q = 2, 3, \ldots, m$.

Correctness of the prefix-function

Corollary 32.7(cont.)

$$
\begin{aligned}
E_{q-1} &= \{k \in \pi^*[q-1] : P[k+1] = P[q]\} \\
&= \{k : k < q \text{ and } P_k \sqsupset P_{q-1} \\
&\qquad \text{and } P[k+1] = P[q]\} \\
&= \{k : k < q-1 \text{ and } P_{k+1} \sqsupset P_q\}
\end{aligned}
$$

Correctness of the prefix-function

Proof.

If E_{q-1} is empty, there is no $k \in \pi^*[q-1]$. Therefore $\pi[q] = 0$.

If E_{q-1} is nonempty, then for each $k \in \pi^*[q-1]$ we have $k+1 < q$ and $P_{k+1} \sqsupset P_q$. Therefore, from the definition of $\pi[q]$, we have

$$\pi[q] \geq 1 + \max\{k \in E_{q-1}\}.$$

Correctness of the prefix-function

Proof.

Note that $\pi[q] > 0$. Let $r = \pi[q] - 1$, so that $r + 1 = \pi[q]$. Since $r + 1 > 0$, we have $P[r + 1] = P[q]$. Furthermore, by Lemma 32.6, we have $r \in \pi^*[q - 1]$. Therefore, $r \in E_{q-1}$, and so

$$\pi[q] \leq 1 + \max\{k \in E_{q-1}\}.$$

□

Correctness of the prefix-function

Correctness of the prefix-function

- At the start of each iteration of the for loop of lines 4-9 in COMPUTE-PREFIX-FUNCTION, we have that $k = \pi[q-1]$.
- The loop on lines 5-6 searches through all values $k \in \pi^*[q-1]$ until one is found for which $P[k+1] = P[q]$; at that point, k is the largest value in the set E_{q-1}, so that, by Corollary 32.7, we can set $\pi[q]$ to $k+1$.

Correctness of the prefix-function

Correctness of the prefix-function

- If no such k is found, $k = 0$ in line 7. If $P[1] = P[q]$, then we should set both k and $\pi[q]$ to 1; otherwise we should leave k alone and set $\pi[q]$ to 0. Lines 7-9 set k and $\pi[q]$ correctly in either case.

Correctness of the KMP algorithm

Correctness of the KMP algorithm

- The procedure KMP-MATCHER can be viewed as a reimplementation of the procedure FINITE-AUTOMATON-MATCHER.
- Specifically, we shall prove that the code in lines 6-9 of KMP-MATCHER is equivalent to line 4 of FINITE-AUTOMATON-MATCHER , which sets q to $\delta(q, T[i])$.

Correctness of the KMP algorithm

Correctness of the KMP algorithm

- Instead of using a stored value of $\delta(q, T[i])$, however, this value is recomputed as necessary from π.
- We claim that $\delta(q, T[i]) = 0$ or $\delta(q, T[i]) - 1 \in \pi^*[q]$.
  - Let $k = \delta(q, T[i])$. Then, $P_k \sqsupset P_q T[i]$ by the definitions of δ and σ.
  - Therefore, either $k = 0$ or else $k \geq 1$ and $P_{k-1} \sqsupset P_q$.

Correctness of the KMP algorithm

Correctness of the KMP algorithm

- Let q' denote the value of q when line 6 is entered.
- Lines 6-9 determine $\delta(q', T[i])$ by examining the elements of $\pi^*[q']$ in decreasing order. When q is found, it means $q = 0$ or $P[q+1] = T[i]$. In the former case, $\delta(q', T[i]) = 0$; in the latter case, q is the maximum element in $E_{q'}$, so that $\delta(q', T[i]) = q + 1$ by Corollary 32.7.

Correctness of the KMP algorithm

Correctness of the KMP algorithm

- Line 12 is necessary in KMP-MATCHER to avoid a possible reference to $P[m+1]$ on line 6 after an occurrence of P has been found.

Idea

Basic idea

More information is gained by matching the pattern from the **right** than from the left.

Observation 1

If current *char* is known not to occur in *pattern*, then we know we need not consider the possibility of an occurrence of *pattern* at *text* positions $1, 2, \ldots,$ or m: Such an occurrence would require that *char* be a character of *pattern*.

Idea

Basic idea

More information is gained by matching the pattern from the **right** than from the left.

Observation 1

If current *char* is known not to occur in *pattern*, then we know we need not consider the possibility of an occurrence of *pattern* at *text* positions $1, 2, \ldots,$ or m: Such an occurrence would require that *char* be a character of *pattern*.

Idea

Observation 2

More generally, if the **last(right-most)** occurrence of *char* in *pattern* is δ_1 characters from the right end of *pattern*, then we know we can slide *pattern* down δ_1 positions without checking for matches.

Idea

Observation 3

When a mismatch occurs at position δ_2 characters from the right end of *pattern*, we can slide to a position to match the *subpattern* $P_{m-\delta_2} \cdots P_m$.

Idea

Example

pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS · −−AT−THAT−POINT···

Since “F” is known not to occur in *pattern*, we can appeal to **Observation 1** and move the pointer down by 7:

Idea

Example

pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS·−−AT−THAT−POINT···

Appealing to **Observation 2**, we can move the pointer down 4 to align the two hyphens:

Idea

Example

pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS · −−AT−THAT−POINT···

Now *char* matches its opposite in *pattern*. Therefore we step left by one:

Idea

Example

pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS·−−AT−THAT−POINT···

Appealing to **Observation 1**, we can move the *pattern* to the right by 6:

Idea

Example

pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS · −−AT−THAT−POINT···

Again *char* matches the last character of *pattern*.
Stepping to the left twice produces:

Idea

Example

pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS·−−AT−THAT−POINT···

Noting that we have a mismatch, we appeal to **Observation 3**. The best move is to align the discovered substring “AT” with the beginning of *pattern*.

Idea

Example

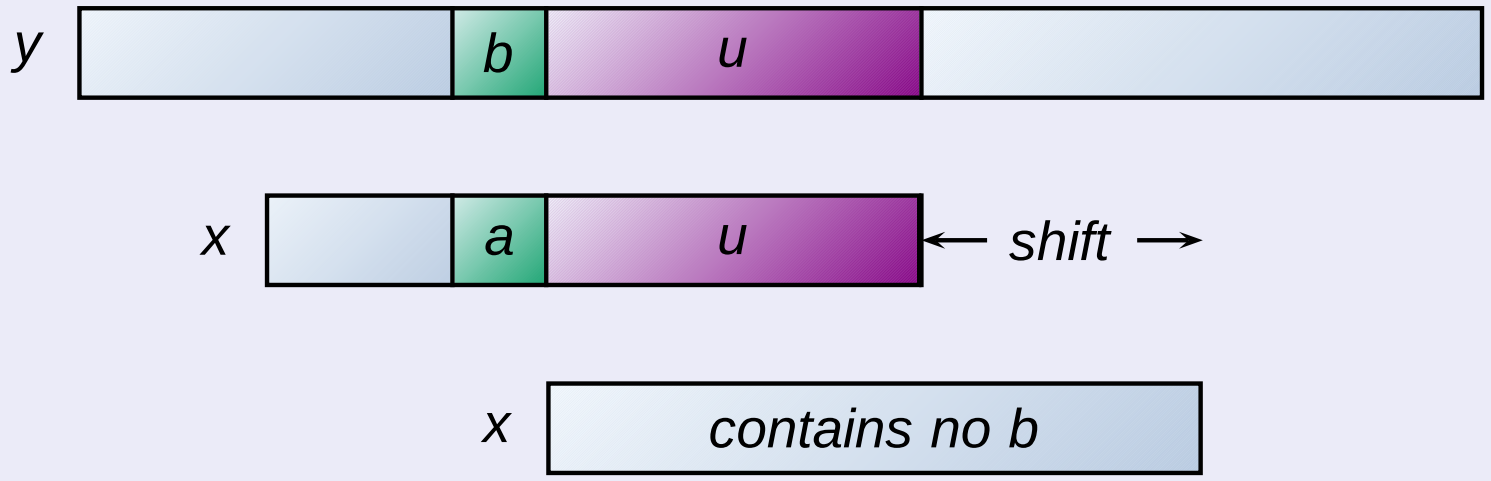
pattern : AT−THAT

text : ···WHICH−FINALLY−HALTS · −−AT−THAT−POINT···

↑

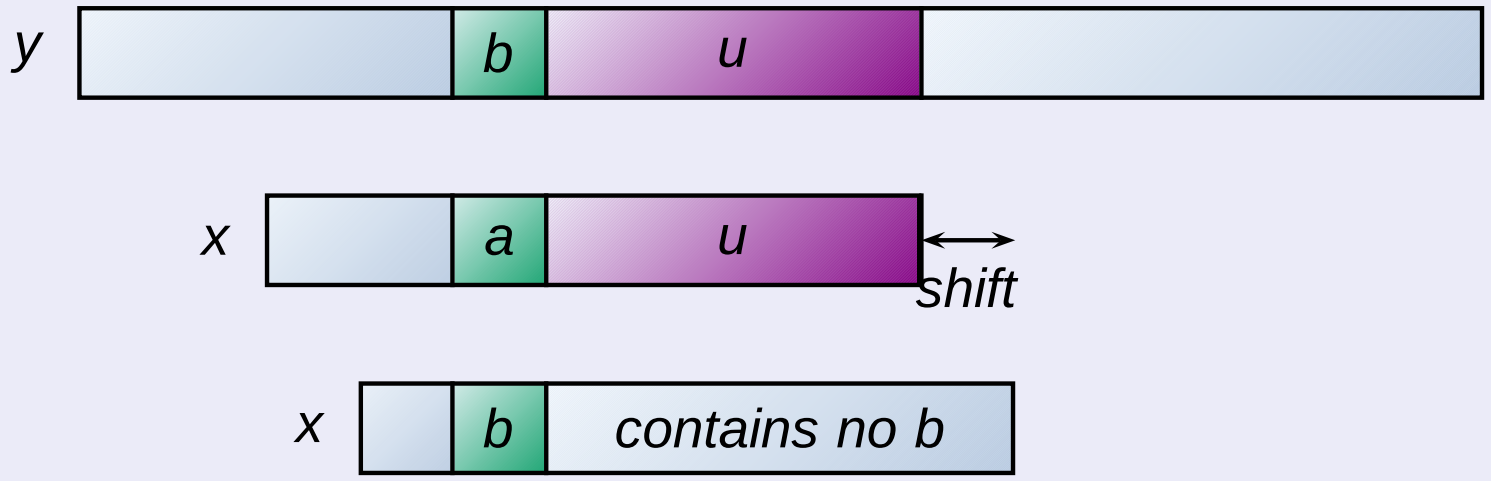
This time we discover the *pattern*. Note that we made only 14 reference to *text*.

Bad-Character Shift



The bad-character shift, b does not occur in x.

Bad-Character Shift



The bad-character shift, b occurs in x.

Bad-Character Shift

Definition

The bad-character shift is stored in a table $bmBc$ of size $|\Sigma|$. For $c \in \Sigma$:

$$bmBc[c] = \begin{cases} \min\{i : 1 \leq i < m-1 \text{ and} \\ \qquad x[m-i] = c\}, & \text{if } c \text{ occurs in } x \\ m, & \text{otherwise.} \end{cases}$$

Bad-Character Shift

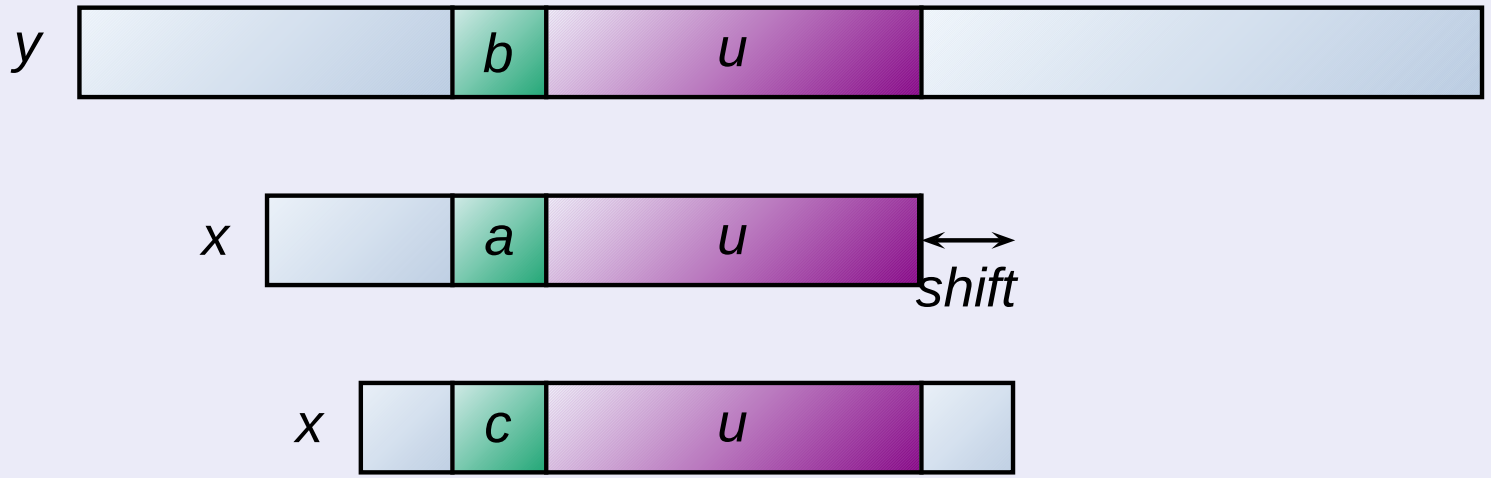
Definition

The bad-character shift is stored in a table $bmBc$ of size $|\Sigma|$. For $c \in \Sigma$:

$$bmBc[c] = \begin{cases} \min\{i : 1 \leq i < m-1 \text{ and} \\ \qquad x[m-i] = c\}, & \text{if } c \text{ occurs in } x \\ m, & \text{otherwise.} \end{cases}$$

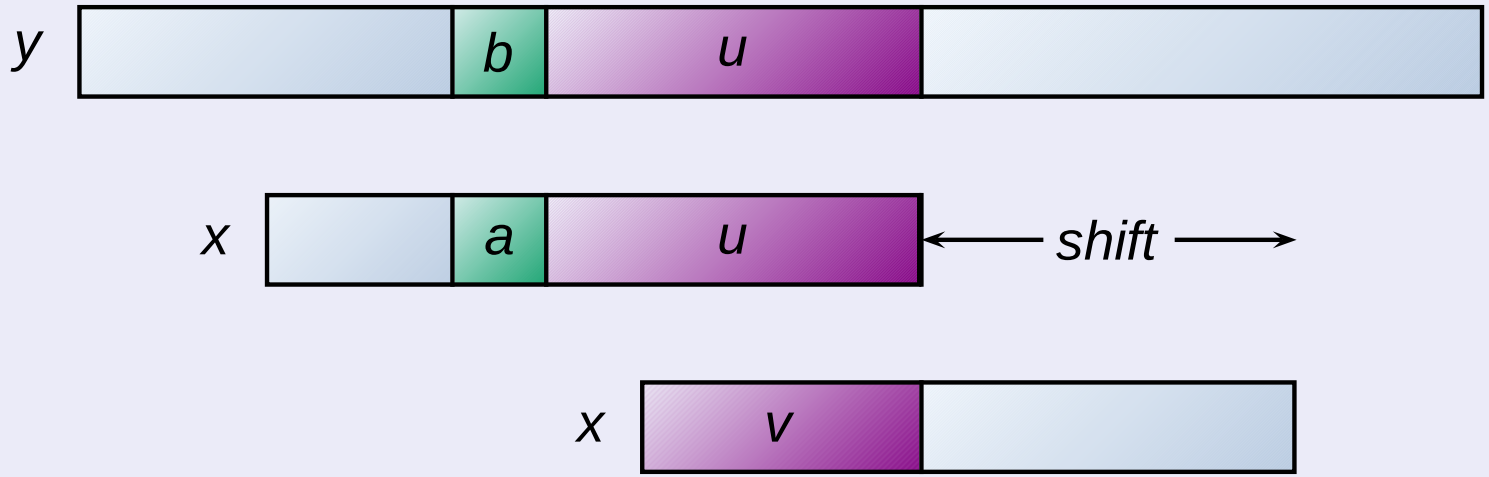
$$shift : s = bmBc[c] - m + i$$

Good-Suffix Shift



The good-suffix shift, u re-occurs preceded by a character c different from a.

Good-Suffix Shift



The good-suffix shift, only a suffix of u re-occurs in x.

Good-Suffix Shift

Definition

The good-suffix shift function is stored in a table *bmGs*.

$$Cs(i,s): \text{ for each } k \text{ such that } i < k \leq m, \\ x[k-s] = x[k] \text{ or } s \geq k$$

and

$$Co(i,s): \text{ if } s < i \text{ then } x[i-s] \neq x[i]$$

Good-Suffix Shift

Definition

Then, for $1 \leq i \leq m$

$$bmGs[i] = \min\{s > 0 : Cs(i, s) \text{ and } Co(i, s) \text{ hold } \}$$

Boyer-Moore Algorithm

BM-MATCHER(T, P)

```
n ← length[T]
m ← length[P]
COMPUTE-BMBC(P, bmBc)
COMPUTE-BMGS(P, bmGs)
s ← 0
```

Boyer-Moore Algorithm

```
ithm 6   while s <= n − m
         do i ← m
            while P[i] = T[s + i]
               do if i = 1
                  then print "Pattern occurs with shift" s
                  else i ← i − 1
            s ← s + MAX(bmGs[i], bmBc[T[s + i]] − m + i)
```

Boyer-Moore Algorithm

COMPUTE-BMBC($P, bmBc$)

```
m ← length[P]
for each character a ∈ Σ
        do bmBc[a] ← m
for i ← 1 to m − 1
        do bmBc[P[i]] ← m − i
```

Boyer-Moore Algorithm

Example

Pattern: GCAGAGAG

$bmBc[A] = 1; bmBc[C] = 6;$
$bmBc[G] = 2; bmBc[T] = 8$

Pattern: ANPANMAN

$bmBc[A] = 1; bmBc[M] = 2;$
$bmBc[N] = 3; bmBc[P] = 5$

How to computer good-suffix shift?

Reverse Suffix Function

$$Rsuff[i] = \max\{k : P[i-k+1..i] = P[m-k+1..m]\}$$

Example

Table: Compute Reverse Suffix

i	1	2	3	4	5	6	7	8
P[i]	G	C	A	G	A	G	A	G
Rsuff[i]	1	0	0	2	0	4	0	8

How to computer good-suffix shift?

Reverse Suffix Function

$$Rsuff[i] = \max\{k : P[i-k+1..i] = P[m-k+1..m]\}$$

Example

Table: Compute Reverse Suffix

i	1	2	3	4	5	6	7	8
P[i]	G	C	A	G	A	G	A	G
Rsuff[i]	1	0	0	2	0	4	0	8

How to computer good-suffix shift?

Reverse Suffix Function

$$Rsuff[i] = \max\{k : P[i-k+1..i] = P[m-k+1..m]\}$$

Example

Table: Compute Reverse Suffix

i	1	2	3	4	5	6	7	8
P[i]	A	N	P	A	N	M	A	N
Rsuff[i]	0	2	0	0	2	0	0	8

How to computer good-suffix shift?

```
COMPUTE-BMGS(P, bmGs)
m ← length[P]
COMPUTE-RSUFF(P, Rsuff)
for i ← 1 to m
        do bmGs[i] ← m
j ← 1
```

How to computer good-suffix shift?

```
for i ← m − 1 downto 1
    do if Rsuff[i] = i
        do while j ≤ m − i
            if bmGs[j] = m
                then bmGs[j] ← m − i
            j ← j + 1
for i ← 1 to m − 1
    do bmGs[m − Rsuff[i]] = m − i
```

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	G	C	A	G	A	G	A	G
Rsuff[i]	1	0	0	2	0	4	0	8
bmGs[i]	8	8	8	8	8	8	8	8
	7	7	7	7	7	7	7	8
	7	7	7	2	7	4	7	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	G	C	A	G	A	G	A	G
Rsuff[i]	1	0	0	2	0	4	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	7	7	7	7	7	7	7	8
	7	7	7	2	7	4	7	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	G	C	A	G	A	G	A	G
Rsuff[i]	1	0	0	2	0	4	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	7	7	7	7	7	7	7	8
	7	7	7	2	7	4	7	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	G	C	A	G	A	G	A	G
Rsuff[i]	1	0	0	2	0	4	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	7	7	7	7	7	7	7	8
	7	7	7	2	7	4	7	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	A	N	P	A	N	M	A	N
Rsuff[i]	0	2	0	0	2	0	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	6	6	6	6	6	6	8	8
	6	6	6	6	6	3	8	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	A	N	P	A	N	M	A	N
Rsuff[i]	0	2	0	0	2	0	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	6	6	6	6	6	6	8	8
	6	6	6	6	6	3	8	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	A	N	P	A	N	M	A	N
Rsuff[i]	0	2	0	0	2	0	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	6	6	6	6	6	6	8	8
	6	6	6	6	6	3	8	1

How to computer good-suffix shift?

Example

Table: Compute bmGs

i	1	2	3	4	5	6	7	8
P[i]	A	N	P	A	N	M	A	N
Rsuff[i]	0	2	0	0	2	0	0	8
	8	8	8	8	8	8	8	8
bmGs[i]	6	6	6	6	6	6	8	8
	6	6	6	6	6	3	8	1

BM Algorithm Example

Example

pattern : GCAGAGAG

text : GCATCGCAGAGAGTATACAGTACG

Shift by 1 ($bmGs[8] = bmBc[A] - 7 + 7 = 1$):

BM Algorithm Example

Example

pattern : GCAGAGAG

text : GCATCGCAGAGAGTATACAGTACG

Shift by 4 ($bmGs[6] = bmBc[C] - 7 + 5 = 4$):

BM Algorithm Example

Example

pattern : GCAGAGAG

text : GCATCGCAGAGAGTATACAGTACG

Shift by 7 ($bmGs[1] = 7$):

BM Algorithm Example

Example

pattern : GCAGAGAG

text : GCATCGCAGAGAGTATACAGTACG

Shift by 4 ($bmGs[6] = bmBc[C] - 7 + 5 = 4$):

BM Algorithm Example

Example

pattern : GCAGAGAG

text : GCATCGCAGAGAGTATACAGTACG

Shift by *bmGs*[7]. The Boyer-Moore algorithm performs 17 text character comparisons on the example.

BM Algorithm History

- It was shown at first that the BM algorithm makes at most $6n$ comparisons if the patter does not occur in the text.
- Guibas and Odlyzko [1980] reduced this to $4n$ under the same assumption.

BM Algorithm History

- Cole[1991] finally proved an essentially tight bound of $3n - \Omega(n/m)$ comparisons for the BM algorithm, whether or not the patter occurs in the text.
- The Turbo BM algorithm takes an additional constant amount of space to complete a search within $2n$ comparisons.

Exercise

Exercise

Compute the bad-character shift and good-suffix shift of pattern “AT-THAT” and “AGAGTAGAG”